Understanding Authentication and Authorization in Technical Documentation

Authentication and authorization are critical concepts for controlling access to applications and their associated resources. Authentication verifies the identity of the entity requesting access, while authorization ensures that the entity is allowed to access specific resources and determines the level of access granted. This document outlines important terms and concepts to understand when implementing authentication and authorization in your application.

The Key Differences

Authentication is the process of verifying who is making the request. It is the first step in ensuring that a user, application, or service is genuine. **Authorization**, on the other hand, defines what resources the authenticated entity can access and the extent of those access rights. **Authentication** is a prerequisite to **authorization**, as determining what can be accessed requires first establishing the identity of the requester.

To illustrate this concept, consider the process of checking into a hotel. Upon arrival, a hotel clerk requests your identification to verify that you made a reservation. This step authenticates your identity. Afterward, the clerk provides you with a key, which authorizes you to access specific areas of the hotel, such as your room, gym, and business center. The hotel key in this example represents your authorization to use those resources.

Process Overview

The following describes the general flow of authentication and authorization in an application:

1. **Configuring the Application**: During the development process, register your app within your cloud console. This registration process involves defining the necessary authorization scopes and setting up credentials, such as an API key, end-user credentials, or service account credentials, to authenticate your app.

2. **Authenticating the App**: When your app is executed, it verifies the registered access credentials. If user authentication is required, a prompt will typically appear for the user to sign in.

3. **Requesting Resources**: When your app requires access to certain resources, it makes a request to the authentication server, specifying the relevant access scopes that were previously registered.

4. **User Consent**: If the app authenticates on behalf of an end-user, a consent screen is presented to the user. This allows the user to decide whether they want to grant the app access to their data or resources.

5. **Sending Approved Requests**: Upon the user's consent, the app sends a request to the authentication server, which includes the approved access scopes and user credentials, to obtain an access token.

6. **Receiving the Access Token**: The server returns an access token, which includes the granted scopes. If the token restricts access to some resources, the app should disable any features or functionality not covered by the token.

7. **Accessing Resources**: The app uses the access token to interact with APIs and access the requested resources.

8. **Obtaining a Refresh Token** (Optional): If the app needs access beyond the life of the initial access token, it may request a refresh token to extend the session.

9. **Requesting Additional Resources**: If additional access is needed, the app can initiate a request for new access scopes, following a similar process to obtain a new access token.

Important Terminology

Below are definitions of key terms related to authentication and authorization:

- **Authentication**: The process of ensuring that a user or app is who they claim to be. For applications that interact with services, there are two main types of authentication:
    - **User Authentication**: The process where an individual user signs into the app, usually through a username and password, to verify their identity.
    - **App Authentication**: Occurs when the app itself authenticates directly with a service or API on behalf of the user, often through pre-created credentials embedded within the app.
- **Authorization**: Refers to the permissions granted to a user or app to access specific data or perform certain actions. This is typically managed by the application through code that requests authorization and, with the user's consent, obtains the appropriate access tokens.

- **Credential**: A piece of information used for security purposes to validate the identity of a user or app. In terms of authentication and authorization, credentials might include API keys, OAuth 2.0 client IDs, or service account keys.
    - **API Key**: A credential used to access public data or non-sensitive resources.
    - **OAuth 2.0 Client ID**: Used for accessing user-owned data, this credential requires the user's consent for the app to access their information.
    - **Client Secret**: A confidential string used alongside the client ID to secure communications. It must be securely stored.
    - **Service Account**: Used for server-to-server interactions, particularly when an app needs to interact with cloud resources or perform automated tasks.
- **Scope**: Defines the level of access or actions the app is permitted to perform. For example, in other APIs, scopes determine which parts of a service the app can access, and users can choose to grant specific scopes.

- **Authorization Server**: The server that grants access to requested data after authentication has been successful, typically by issuing an access token.

- **Access Token**: A credential used to gain access to protected resources. The token is typically issued by an authorization server and includes the scope of access granted.
- **OAuth 2.0 Framework**: A widely used protocol that provides secure delegated access, allowing an app to perform actions on behalf of the user without exposing sensitive credentials.
- **Principal**: Any entity that can be granted access to a resource. This can include user accounts or service accounts.
- **Data Types**: The types of data your app might attempt to access, categorized as public domain data, end-user data, or cloud data.
- **User Consent**: The step where users grant permission for the app to access data and perform actions on their behalf within the defined scope.
- **Application Type:** Defines the type of app (e.g., web, mobile, desktop) during the credentials setup process.
- **Service Account:** An account that authenticates non-human users (like apps) to access data and perform actions on behalf of your app, often in server-to-server interactions.
- **Domain Delegation:** Allows apps to access user data across an organization, typically used for administrative tasks, with access granted by super administrators.

Configuring the OAuth Consent Screen and Defining Scopes

When using OAuth 2.0 for authorization, a consent screen is displayed to the user, providing essential details about your project, its policies, and the requested authorization scopes. Configuring your app's OAuth consent screen ensures that users and reviewers receive the necessary information and allows your app to be registered for future publishing.

It's important to review any API-specific documentation related to authentication and authorization before proceeding.

Defining Authorization Scopes

Authorization scopes define the level of access your app requests from users. These scopes are structured as URI strings, specifying the app's name, the data it will access, and the level of access required. Scopes detail the permissions your app needs to interact with data, including user account data.

When your app is installed, users will need to confirm the requested scopes. It's important to select the most limited and relevant scope to ensure users are more likely to grant access to only the necessary permissions.

OAuth 2.0 consent screens are required for all apps, but scopes need to be listed only for apps that are used by individuals outside of your organization.

Tip: If you are unsure about the required consent screen details, you can use placeholder information until your app is ready for release.

For security reasons, once the OAuth 2.0 consent screen is configured, it cannot be removed.

Steps for Configuring OAuth Consent

1. In the platform console, navigate to **Menu > Auth Platform > Branding**.
2. If the Auth platform is already configured, you can update settings under **Branding**, **Audience**, and **Data Access**. If it's not configured, click **Get Started**.
3. In the **App Information** section:
    - Enter your **App name**.
    - For **User support email**, select the email address where users can contact you for consent-related inquiries.
    - Click **Next**.
4. Under the **Audience** section, specify the user type for your app and click **Next**.
5. Under **Contact Information**, enter an email address to receive notifications about any changes to your project, then click **Next**.
6. Review the User Data Policy. If you agree, select the consent option, then click **Continue**.
7. Click **Create**.
8. If you selected **External** for the user type, add test users by navigating to **Audience > Test users** and clicking **Add users**. Enter your email and any authorized test users, then click **Save**.
9. If your app is designed for external use (outside your organization), go to **Data Access > Add or Remove Scopes**. Follow best practices when selecting scopes:
    - Only select the scopes your app truly needs.
    - For a list of available scopes, refer to the OAuth 2.0 documentation.
    - Review the categories of scopes: non-sensitive, sensitive, and restricted. Where possible, choose non-sensitive scopes to avoid extra reviews.
    - Sensitive or restricted scopes may require additional reviews, especially for external apps. Internal apps can use sensitive or restricted scopes without requiring further review.

Once you've selected the necessary scopes, click **Save**.

For additional guidance on configuring OAuth consent, refer to the platform documentation.

Scope Categories

Different levels of access associated with scopes have varying requirements:

Scope Type	Description	Basic App Verification	Additional App Verification	Security Assessment
Non-sensitive scopes	These grant access to limited data relevant to specific	Check	—	—
Sensitive scopes	These grant access to personal user data or sensitive resources.	Check	Check	—
Restricted scopes	These provide access to highly-sensitive or extensive user data	Check	Check	Check

Next Steps:

After configuring the OAuth consent screen and selecting the required scopes, the next step is to create the access credentials for your app.

Create Access Credentials for Your App

Credentials are essential for obtaining an access token from an authorization server, allowing your app to interact with APIs. This guide provides instructions on selecting and configuring the correct credentials for your app.

Choose the Right Access Credential

The credential type required depends on the data, platform, and access needs of your app. There are three main types of credentials:

Use Case	Authentication	About This Authentication Method
Access publicly available data anonymously.	**API Key**	This method is used to access publicly available data. Check the API documentation to confirm whether it
Access user data, such as email or age.	**OAuth Client ID**	Requires user consent to authenticate and access their data.
Access data within your application or on behalf of	**Service Account**	Grants an app access to resources or data associated with a service account.

API Key Credentials

API keys are used for accessing publicly available data. They are a long string containing letters, numbers, underscores, and hyphens. You can use them to access resources that do not require user authentication.

Steps to Create an API Key:

1. Navigate to **Credentials** in the management console.
2. Click **Create credentials** and select **API key**.
3. Your API key will be displayed. Click **Copy** to use it in your app's code.
4. To restrict the use of the API key, click **Restrict key** and set the necessary restrictions.

OAuth Client ID Credentials

If your app needs to access user data, you will require OAuth client IDs to authenticate users. This method involves requesting consent from users to access their data.

Steps to Create an OAuth Client ID:

1. Go to **Credentials**.
2. Click **Create Client**.
3. Choose the application type (e.g., Web application, Android, iOS).
4. Add the required **authorized URIs**:
    - For client-side apps, add **JavaScript origins**.
    - For server-side apps, add **redirect URIs**.
5. Click **Create**.
6. The newly created OAuth client ID will appear under **OAuth 2.0 Client IDs**.

Note: Client secrets are not required for web applications.

Service Account Credentials

A service account allows your app to authenticate as a special user, granting it access to data or actions on behalf of other users, especially in domains or organizations.

Steps to Create a Service Account:

1. Go to **Service Accounts**.
2. Click **Create service account**.
3. Fill in the service account details and click **Create**.
4. Optionally, assign roles to the service account to control access to project resources.
5. Click **Continue**, then **Done**. Make a note of the service account's email address.

Assigning Roles to a Service Account:

1. In the **Admin console**, go to **Account > Admin roles**.
2. Select the role to assign and click **Assign service accounts**.
3. Enter the service account's email address and click **Add > Assign role**.

Service Account Credentials: Key Pair Creation

To use a service account, you need to create a public/private key pair for authentication.

Steps to Create Service Account Credentials:

1. In the management console, go to **Service Accounts**.
2. Select your service account.
3. Click **Keys > Add key > Create new key**.
4. Choose **JSON** and click **Create**.
5. Your new key pair will be downloaded as a file. Save it securely in your project directory as **credentials.json**.

Note: Keep the key file secure and ensure it is not exposed publicly.

Optional: Set Up Domain-Wide Delegation

If your app needs to call APIs on behalf of users within a domain, you must configure domain-wide delegation for the service account.

Steps to Set Up Domain-Wide Delegation:

1. In the **Service Accounts** section, select the service account and click **Show advanced settings**.
2. Copy the **Client ID** and go to the **Admin console**.
3. In the Admin console, go to **Security > Access and data control > API controls**.
4. Click **Manage Domain-Wide Delegation** and then **Add new**.
5. Paste the **Client ID** into the field and list the required OAuth Scopes.
6. Click **Authorize**.

Next Steps:

Once you've set up your credentials, you can proceed with integrating them into your app and calling the appropriate APIs.